

FIC_DEV
PROGRAMADOR DE SISTEMAS COM IA

DESAFIO PRÁTICO 1

Pipeline de Recomendação e Dashboard de Conteúdos Educacionais

Fundamentos de Dados para IA

Carga horária recomendada: 12 horas
Modalidade: equipe de três estudantes
Versão: 1.0 — 2026

1. Orientações gerais

Este desafio integra os conteúdos das aulas 1 a 8 do curso Fundamentos de Dados para IA. A equipe deverá construir um fluxo completo de dados, desde a ingestão e o armazenamento até a recomendação e a apresentação de indicadores em um dashboard.

- Leia integralmente o enunciado antes de iniciar a implementação.
- Utilize dados fictícios e preserve os arquivos originais de entrada.
- Mantenha a solução dentro do escopo proposto para que seja concluída em aproximadamente 12 horas.
- Registre decisões, limitações e instruções de execução no README.md.
- As funcionalidades adicionais não substituem requisitos obrigatórios ausentes.
- Todos os integrantes deverão compreender a solução completa.

2. Objetivos de aprendizagem

Ao concluir o desafio, espera-se que o discente seja capaz de:

- ingerir e validar dados provenientes de diferentes fontes e formatos;
- modelar e armazenar dados estruturados em um banco relacional;
- armazenar e consultar dados semiestruturados em um banco NoSQL;
- representar descrições textuais por meio de embeddings;
- realizar busca por similaridade semântica;
- implementar uma estratégia simples de recomendação;
- definir métricas e KPIs orientados à tomada de decisão;
- construir visualizações e um dashboard no Apache Superset;
- Aplicar práticas básicas de DataOps e documentar o uso de Inteligência Artificial.

3. Situação-problema

Uma plataforma fictícia disponibiliza cursos, vídeos, artigos, podcasts e outros materiais educacionais. Atualmente, os dados estão distribuídos em diferentes arquivos e formatos, dificultando a identificação dos conteúdos mais procurados, a análise do comportamento dos usuários, a

avaliação da qualidade dos materiais, a recomendação de conteúdos relacionados e a construção de indicadores para apoiar decisões.

A instituição deseja organizar esses dados, produzir recomendações simples e acompanhar os principais resultados em um dashboard.

Produto esperado: pipeline reproduzível que integre ingestão, PostgreSQL, MongoDB, armazenamento vetorial, recomendações e um dashboard no Apache Superset.

4. Problema proposto

Cada equipe deverá desenvolver uma solução de dados capaz de:

1. ingerir dados provenientes de diferentes fontes e formatos;
2. armazenar dados estruturados em um banco relacional;
3. armazenar dados semiestruturados em um banco NoSQL;
4. representar descrições textuais por meio de embeddings;
5. recuperar conteúdos semanticamente semelhantes;
6. gerar recomendações simples;
7. disponibilizar métricas e KPIs em um dashboard no Apache Superset.

A solução deverá formar um fluxo integrado e reproduzível, desde a leitura dos dados de origem até sua apresentação no dashboard.

5. Organização das equipes

O desafio deverá ser realizado por equipes de 3 (três) estudantes, NO MÁXIMO. A divisão das responsabilidades fica a cargo da equipe. Uma sugestão para divisão inicial de responsabilidades é:

Integrante	Responsabilidade inicial
Estudante 1	Ingestão, tratamento e PostgreSQL.
Estudante 2	MongoDB, embeddings e recomendações.
Estudante 3	Métricas, consultas e dashboard no Superset.

Importante: a divisão é apenas uma orientação de trabalho. Todos os integrantes deverão compreender e saber explicar a solução completa.

6. Fontes de dados

As equipes receberão ou deverão produzir dados fictícios organizados em, pelo menos, três fontes.

6.1 Catálogo de conteúdos — CSV

Campo	Descrição
conteudo_id	Identificador do conteúdo.
titulo	Título do material.

tipo	Curso, vídeo, artigo ou podcast.
categoria	Área temática.
nível	Básico, intermediário ou avançado.
carga_horaria_min	Duração estimada em minutos.
data_publicacao	Data de publicação.
descricao	Descrição textual.
autor	Responsável pelo conteúdo.

6.2 Interações dos usuários — JSON

A fonte deverá conter, no mínimo, identificador do usuário, identificador do conteúdo, tipo de interação, data e hora, tempo consumido, percentual de conclusão e avaliação atribuída.

Tipos de interação sugeridos:

- visualização;
- início;
- conclusão;
- curtida;
- avaliação;
- compartilhamento.

6.3 Comentários e avaliações — JSON ou MongoDB

```
Exemplo de documento: {
  "usuario_id": 104,
  "conteudo_id": 28,
  "avaliacao": 5,
  "comentario": "Conteúdo introdutório, claro e objetivo.",
  "tags": ["didático", "iniciante", "python"],
  "data": "2026-08-20"
}
```

7. Requisitos funcionais obrigatórios

O sistema deverá atender aos requisitos funcionais apresentados nesta seção. Cada requisito deverá ser implementado, testado e demonstrado pela equipe.

RF01 — Inicialização e configuração

O sistema deverá:

- ser executado pelo comando:

```
python -m src.main
```

- utilizar um arquivo de configuração no formato JSON ou YAML;

- permitir configurar os caminhos dos arquivos de entrada;
- permitir configurar os parâmetros de conexão com o PostgreSQL, o MongoDB e o banco vetorial;
- separar senhas e informações sensíveis do código-fonte;
- apresentar mensagens que indiquem o início e o término do processamento.

As senhas poderão ser armazenadas em variáveis de ambiente ou em um arquivo `.env` que não deverá ser incluído no controle de versão.

RF02 — Leitura das fontes de dados

O sistema deverá ler, no mínimo:

- um arquivo CSV contendo o catálogo de conteúdos educacionais;
- um arquivo JSON contendo as interações dos usuários;
- um arquivo JSON contendo comentários ou avaliações.

O sistema deverá informar o nome e a quantidade de registros encontrados em cada fonte.

RF03 — Validação dos dados

Cada registro deverá ser classificado como:

- válido;
- inválido;
- incompleto;
- duplicado.

A aplicação deverá registrar o motivo da classificação dos registros que não forem considerados válidos.

As validações deverão contemplar, quando aplicável:

- presença dos campos obrigatórios;
- validade dos identificadores;
- formato das datas;
- domínio dos valores categóricos;
- intervalo das avaliações;
- valores numéricos negativos ou incompatíveis;
- referências a usuários ou conteúdos inexistentes.

RF04 — Tratamento e padronização

O sistema deverá:

- remover espaços desnecessários;
- uniformizar o uso de letras maiúsculas e minúsculas;
- padronizar categorias, tipos e níveis dos conteúdos;
- converter datas para um formato padronizado;
- converter campos numéricos;

- tratar ou registrar valores ausentes;
- identificar e eliminar duplicidades;
- preservar os arquivos originais;
- gravar os dados tratados em um diretório próprio.

As decisões de tratamento deverão ser registradas na documentação da solução.

RF05 — Resumo da ingestão

Ao final da ingestão, o sistema deverá apresentar e armazenar um resumo contendo:

- quantidade de registros lidos;
- quantidade de registros válidos;
- quantidade de registros inválidos;
- quantidade de registros incompletos;
- quantidade de registros duplicados;
- quantidade de registros corrigidos;
- quantidade de registros carregados em cada banco de dados;
- tempo total de processamento.

O resumo deverá ser gravado em formato JSON.

RF06 — Persistência no PostgreSQL

O sistema deverá armazenar os dados estruturados no PostgreSQL.

O banco deverá possuir, no mínimo, entidades equivalentes a:

- usuário;
- conteúdo;
- categoria;
- interação;
- recomendação.

A implementação deverá:

- definir chaves primárias;
- definir chaves estrangeiras;
- impedir duplicidades de identificadores;
- manter a integridade dos relacionamentos;
- utilizar transações durante a carga;
- permitir consultar os registros armazenados.

A equipe deverá entregar o script SQL utilizado para criar as tabelas.

RF07 — Persistência no MongoDB

O sistema deverá armazenar no MongoDB os comentários, avaliações ou outros dados semiestruturados.

Cada documento deverá manter a identificação do usuário e do conteúdo ao qual está relacionado.

A aplicação deverá permitir, no mínimo:

- inserir documentos;
- consultar comentários de determinado conteúdo;
- localizar documentos por tag;
- filtrar avaliações pela nota;
- agregar a quantidade de comentários ou avaliações por categoria.

A equipe deverá justificar a escolha dos dados armazenados no MongoDB.

RF08 — Geração e armazenamento de embeddings

O sistema deverá:

- utilizar o título e a descrição dos conteúdos para gerar uma representação textual;
- gerar um embedding para cada conteúdo válido;
- associar o embedding ao identificador do conteúdo;
- armazenar os vetores em PostgreSQL com `pgvector`,
- evitar a geração duplicada de embeddings para o mesmo conteúdo;
- registrar o modelo utilizado para gerar os vetores.

A equipe deverá documentar o modelo de embeddings e a estratégia de preparação dos textos.

RF09 — Busca por similaridade semântica

O sistema deverá receber uma consulta em linguagem natural e retornar os conteúdos semanticamente mais semelhantes.

Exemplo de consulta:

❑ Quero aprender os fundamentos de banco de dados para inteligência artificial.

❑ Para cada resultado, o sistema deverá apresentar:

- posição no resultado;
- identificador do conteúdo;
- título;
- categoria;
- tipo;
- valor de similaridade ou distância.

A quantidade de resultados retornados deverá ser configurável. A equipe deverá demonstrar pelo menos três consultas semânticas diferentes.

RF10 — Geração de recomendações

O sistema deverá gerar recomendações de conteúdos para um usuário.

A recomendação deverá considerar, no mínimo:

- conteúdos visualizados;
- conteúdos curtidos ou bem avaliados;
- remoção dos conteúdos já concluídos pelo usuário.

O sistema deverá apresentar, para cada recomendação:

- usuário;
- conteúdo recomendado;
- pontuação;
- posição;
- data de geração;

A equipe deverá utilizar a seguinte fórmula:

A formulação baseia-se exclusivamente em três variáveis calculadas entre o usuário e o conteúdo candidato:

- **Índice de Visualizações (Ivis):** Mede o nível de afinidade temática com os conteúdos que o usuário consome e visualiza com frequência (calculado por similaridade vetorial via pgvector ou pela proporção de tempo consumido na mesma categoria), assumindo valores contínuos na faixa de 0.0 a 1.0.
- **Índice de Curtidas e Avaliações (Icur):** Representa o interesse explícito e a aprovação do usuário em materiais relacionados (baseado no histórico de curtidas ou avaliações positivas com nota igual ou superior a 4), variando igualmente entre 0.0 e 1.0.
- **Índice de Remoção de Concluídos (Iconc):** Funciona como um filtro binário eliminatório obrigatório pelo requisito de negócio. Ele assume valor 0 caso o usuário já tenha concluído o conteúdo (anulando qualquer pontuação), e valor 1 caso o material ainda não tenha sido concluído, mantendo-o apto para recomendação.

$$\text{Pontuação} = ((Ivis + Icur)/2) * 100 * Iconc$$

Regra básica: A pontuação é a média simples entre o índice de visualização e o de curtidas, multiplicada pelo filtro de conclusão. Varia de 0 a 100.

3. Regra de Classificação (Tipo de Recomendação)

Com base na pontuação calculada, o sistema atribui o status da recomendação:

- **Positivo (Pontuação >= 70):**
Forte afinidade. O usuário visualiza ativamente conteúdos desse tema e costuma curtir/avaliar positivamente.
- **Estável (40 > Pontuação < 70):**
Afinidade moderada. O usuário demonstrou interesse parcial (ou apenas visualizou pouco, ou ainda não avaliou conteúdos semelhantes).
- **Negativo (Pontuação <= 40 ou Iconc= 0):**
Baixo interesse ou conteúdo já concluído (descartado da lista de sugestões).

RF11 — Persistência das recomendações

As recomendações geradas deverão ser armazenadas no PostgreSQL.

Cada recomendação deverá possuir, no mínimo:

- identificador do usuário;
- identificador do conteúdo;
- pontuação final;
- posição no resultado;
- data e hora da geração.

RF12 — Produção de métricas e KPIs

O sistema deverá calcular, no mínimo:

- duas métricas operacionais;
- dois KPIs orientados à tomada de decisão.

Entre os indicadores possíveis estão:

- quantidade de usuários;
- quantidade de conteúdos;
- visualizações por período;
- avaliação média;
- taxa de conclusão;
- engajamento;
- retenção;
- conversão de recomendações;
- tempo médio consumido;
- quantidade de recomendações geradas.

Para cada KPI, a equipe deverá informar:

- nome;
- objetivo;
- fórmula;
- fonte dos dados;
- periodicidade;
- interpretação.

Os resultados deverão ser disponibilizados em tabelas ou visões no PostgreSQL para utilização pelo Apache Superset.

RF13 — Dashboard no Apache Superset

A equipe deverá construir um dashboard no Apache Superset utilizando os dados consolidados no PostgreSQL.

O dashboard deverá conter, no mínimo:

- três cartões de indicadores;
- um gráfico de barras;
- um gráfico de linhas;
- dois filtros interativos.

O dashboard deverá permitir responder a pelo menos duas perguntas de negócio definidas pela equipe.

A escolha de cada gráfico deverá ser justificada considerando a natureza dos dados e a informação que se deseja comunicar.

RF14 — Registro de execução

Durante a execução, o sistema deverá registrar:

- início e término do processamento;
- arquivos processados;
- quantidade de registros lidos;
- registros rejeitados;
- falhas de conexão;
- falhas na geração de embeddings;
- falhas de persistência;
- tempo de execução das principais etapas.

O registro deverá permitir identificar a origem e a causa provável de cada problema.

8. Arquitetura mínima esperada

Etapa	Tecnologia ou resultado
Fontes	CSV, JSON
Ingestão	Leitura, validação, limpeza e registro de erros.
Dados estruturados	PostgreSQL.
Dados semiestruturados	MongoDB.
Dados vetoriais	pgvector.
Processamento	Busca semântica e motor de recomendação.
Apresentação	Apache Superset conectado aos dados consolidados no PostgreSQL.

A Figura 1 é uma representação gráfica da arquitetura mínima esperada para a solução do desafio.

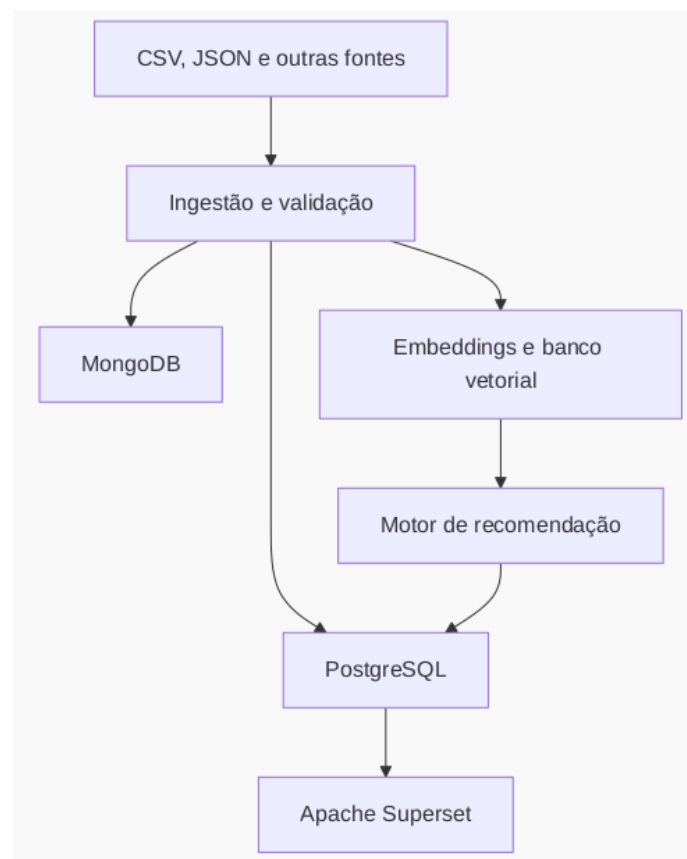


Figura 1. Representação gráfica da arquitetura.

9. Práticas de DataOps

A solução deverá apresentar práticas básicas de DataOps:

- código organizado em diretórios;
- arquivo README.md;
- dependências registradas;

- parâmetros de conexão separados do código;
- registro das etapas executadas;
- tratamento básico de erros;
- instruções para reproduzir a solução;
- controle de versão, quando possível.

10. Uso da Inteligência Artificial

A IA poderá ser utilizada para explicar erros, sugerir comandos SQL, auxiliar na modelagem, produzir expressões regulares, propor tratamentos para dados ausentes, revisar código, sugerir métricas, apoiar a geração de embeddings e melhorar a documentação.

A equipe continuará responsável por testar, compreender e justificar tudo o que entregar. Como parte da documentação, deverá apresentar um pequeno registro contendo:

- ferramenta de IA utilizada;
- exemplos de solicitações realizadas;
- trechos ou decisões apoiadas pela IA;
- erros ou inadequações encontrados nas respostas;
- alterações feitas pela equipe.

11. Entregáveis

Cada equipe deverá entregar um arquivo compactado com a seguinte organização sugerida:

```

Estrutura: desafio_dados/
├── README.md
├── dados/
│   ├── brutos/
│   └── processados/
├── ingestao/
├── recomendacao/
├── sql/
│   ├── criar_banco.sql
│   └── consultas.sql
├── mongodb/
│   └── consultas.js
├── dashboard/
│   └── evidencias/
├── documentacao/
│   ├── modelo_de_dados.pdf
│   ├── arquitetura.pdf
│   ├── kpis.md
│   └── uso_da_ia.md
└── requirements.txt
  
```

Entregáveis obrigatórios:

1. código de ingestão;
2. arquivos de dados utilizados;
3. modelo conceitual e lógico;
4. scripts do PostgreSQL;
5. documentos e consultas do MongoDB;
6. implementação da busca vetorial;

7. implementação da recomendação;
8. exportação do dashboard do Superset;
9. documentação para instalação e execução;

12. Apresentação

Cada equipe deverá realizar uma apresentação de no **máximo** 10 minutos:

Tempo	Conteúdo
2 minutos	Problema e arquitetura.
3 minutos	Ingestão e armazenamento.
2 minutos	Busca vetorial e recomendação.
2 minutos	Dashboard e principais descobertas.
1 minuto	Dificuldades, limitações e uso da IA.

Verificação de aprendizagem: qualquer integrante poderá ser solicitado a explicar uma parte da solução.